

```

import pandas as pd
import time
import random
import re

# Automatically ensure cloudscraper is installed to maximize bypass potential
try:
    import cloudscraper
except ImportError:
    import os
    os.system('pip install cloudscraper')
    import cloudscraper

# -----
# STEP 1: PASTE YOUR COMPLETED MOVIE LIST HERE
# -----
master_movie_list = [
    "Roja", "Pudhiya Mugam", "Gentleman", "Kizhakku Cheemayile", "Uzhavan",
    "Thiruda Thiruda", "Vandicholai Chinraasu", "Duet", "May Madham", "Kadhalan",
    "Pavithra", "Karuththamma", "Bombay", "Indira", "Love Birds", "Indian",
    "Kadhal Desam", "Mr. Romeo", "Love Birds", "Minsara Kanavu", "Iruvar",
    "Ratchagan", "Jeans", "En Swasa Kaatre", "Padayappa", "Kadhalar Dhinam",
    "Sangamam", "Jodi", "Mudhalvan", "Taj Mahal", "Alaipayuthey",
    "Kandukondain Kandukondain", "Rhythm", "Thenali", "Star", "Parthale Paravasam",
    "Alli Arjuna", "Kannathil Muthamittal", "Baba", "Boys", "Ennakku 20 Unnakku 18",
    "Aaytha Ezhuthu", "New", "Anbe Aaruyire", "Sillunu Oru Kaadhal", "Varalaru",
    "Sivaji", "Azhagiya Tamil Magan", "Sakkarakatti", "Vinnaihaandi Varuvaayaa",
    "Raavanan", "Enthiran", "Koaadadal", "Maryan", "Kadal", "Kaaviya Thalaivan",
    "Lingaa", "O Kadhal Kanmani", "Achcham Yenbadhu Madamaiyada", "24",
    "Kaatru Veliyidai", "Mersal", "Chekka Chivantha Vaanam", "Sarkar", "2.0",
    "Bigil", "Vendhu Thanindhathu Kaadu", "Ponniyin Selvan: I", "Ponniyin Selvan: II",
    "Maamannan", "Lal Salaam", "Thug Life"
]

database_records = []

# Keywords unique to Romanised Tamil (Tanglish) to filter out Hindi/Telugu duplicates
tamil_identifiers = ["kaadhal", "kanmani", "unnai", "ennai", "penn", "vandu", "mazhai", "illai", "nee", "naan", "thaan", "th

print(f"📁 Loaded custom master list tracking {len(master_movie_list)} films.")
print("🚀 Launching Self-Healing Cloud-Bypass Scraper Loop...\n")

scraper = cloudscraper.create_scraper()

for i, movie in enumerate(master_movie_list, 1):
    url = "https://lrclib.net/api/search"
    query_parameters = {"q": f"A.R. Rahman {movie}"}

    print(f"🕒 [{i}/{len(master_movie_list)}] Querying: '{movie}'...")

    try:
        # Request with fake real-browser connection configurations
        response = scraper.get(url, params=query_parameters, timeout=12)

        if response.status_code == 200:
            # CRUCIAL PROTECTION LAYER: Only attempt JSON parse if text layout looks right
            # If server spits out an HTML wall, this prevents a program crash
            if response.text.strip().startswith("<DOCTYPE") or not response.text.strip().startswith("["):
                print(f"⚠️ Cloudflare firewall blocked '{movie}' with an HTML page. Skipping safely...")
                continue

            try:
                tracks = response.json()
                found_songs_in_movie = 0

                if tracks and len(tracks) > 0:
                    for track in tracks:
                        track_title = track.get("trackName", "")
                        plain_lyrics = track.get("plainLyrics", "")

                        if not plain_lyrics:
                            continue

                        # FILTER 1: REMOVE ALL NATIVE SCRIPTS (தமிழ், हिंदी, etc.)
                        contains_native_script = re.search(r'[\u0900-\u097F\u0B80-\u0BFF\u0C00-\u0C7F]', plain_lyrics)
                        if contains_native_script:
                            continue

                        # FILTER 2: CONFIRM ROMANISED TAMIL TEXT
                        lyrics_lower = plain_lyrics.lower()
                        is_tamil = any(word in lyrics_lower for word in tamil_identifiers)

```

```

is_tamil = any(word in lyrics_lower for word in tamil_keywords)

if is_tamil:
    # Avoid duplicates inside dataset
    if not any(d['Song Title'] == track_title and d['Movie/Album'] == movie for d in database_records):
        database_records.append({
            "Song Title": track_title,
            "Movie/Album": movie,
            "Composer": "A.R. Rahman",
            "Language": "Tamil (English Script)",
            "Lyrics": plain_lyrics
        })
        found_songs_in_movie += 1

if found_songs_in_movie > 0:
    print(f" 🟢 Success! Saved {found_songs_in_movie} clean Romanised tracks.")
else:
    print(f" 🚫 Skipped (Lyrics are either missing, or in native script).")

except ValueError:
    print(f" ⚠️ Data parsing exception caught on response text for '{movie}'. Skipping context...")

elif response.status_code == 429:
    print(f" 🛑 Hit rate limiter threshold! Taking a heavy break...")
    time.sleep(15)
else:
    print(f" ❌ Server rejected endpoint request. Code: {response.status_code}")

except Exception as error:
    print(f" 🌐 Network gateway connection dropped for '{movie}': {error}")

# Randomized timing loop intervals to lower threat score flags on server
time.sleep(random.uniform(3.0, 5.5))

# -----
# STEP 3: EXPORT RECOVERED TRACKS
# -----
print("\n-----")
if len(database_records) > 0:
    lyrics_dataframe = pd.DataFrame(database_records)
    output_file = "rahman_complete_english_tamil.csv"
    lyrics_dataframe.to_csv(output_file, index=False, encoding="utf-8-sig")
    print(f" 🎉 Complete! Process executed successfully without crashing.")
    print(f" 📁 Saved as: '{output_file}'")
    print(f" 🎵 Total Unique Romanised Tamil Songs Saved: {len(lyrics_dataframe)}")
else:
    print(f" 🚨 The script executed, but your current network IP is completely blacklisted by their firewall cluster.")
    print(f" 💡 FIX: Turn on a VPN, connect to your phone's mobile hotspot, or copy this code into a free Google Colab file to")

```

```

[65/72] Querying: '2.0'...
[66/72] Querying: 'Bigil'...
[67/72] Querying: 'Vendhu Thanindhathu Kaadu'...
[68/72] Querying: 'Ponniyin Selvan: I'...
[69/72] Querying: 'Ponniyin Selvan: II'...
[70/72] Querying: 'Maamannan'...
[71/72] Querying: 'Lal Salaam'...
[72/72] Querying: 'Thug Life'...
-----
Complete! Process executed successfully without crashing.
Saved as: 'rahman_complete_english_tamil.csv'
Total Unique Romanised Tamil Songs Saved: 154

```

```

import sys
import subprocess
import re

# =====
# LINE 1: AUTOMATIC DEPENDENCY CHECKER
# =====
# This safely installs ChromaDB and SentenceTransformers if they are missing
try:
    import chromadb
    from sentence_transformers import SentenceTransformer
    import pandas as pd
except ImportError:
    print("🔧 Missing libraries detected. Installing ChromaDB and Transformers for free...")
    subprocess.check_call([sys.executable, "-m", "pip", "install", "chromadb", "sentence-transformers", "pandas"])
    import chromadb
    from sentence_transformers import SentenceTransformer
    import pandas as pd

# =====
# LINE 2: LOAD YOUR SCRAPED TAMIL LYRICS FILE
# =====
# Make sure your file name matches exactly what was saved earlier
df = pd.read_csv("rahman_complete_english_tamil.csv")

# =====
# LINE 3: TRANSFORMATION ENGINE FUNCTION
# =====
def clean_lyric_text(text):
    if not isinstance(text, str):
        return ""

    # 1. Strip out 'Male:', 'male:', 'Female:', 'female:' case-insensitively
    text_no_tags = re.sub(r'(?i)(male|female):\s*', '', text)

    # 2. Split into individual lines to keep the song structure intact
    lines = text_no_tags.split('\n')
    cleaned_lines = []

    for line in lines:
        words = line.split()
        word_counts = {}
        line_result = []

        for word in words:
            # Clean punctuation from words to count them accurately
            clean_word = re.sub(r'^\w', '', word.lower())
            word_counts[clean_word] = word_counts.get(clean_word, 0) + 1

            # CRUCIAL CONSTRAINT: Drop any word repeating more than 2 times in a row/line
            if word_counts[clean_word] >= 2:
                line_result.append(word)

        cleaned_lines.append(" ".join(line_result))

    return "\n".join(cleaned_lines)

# =====
# LINE 4: PROCESS AND CLEAN THE ENTIRE FILE
# =====
df['Cleaned_Lyrics'] = df['Lyrics'].apply(clean_lyric_text)

```

```

# =====
# LINE 5: INITIALIZE THE LOCAL CHROMADB STORE
# =====
# This sets up a permanent database folder named 'rahman_vector_db'
chroma_client = chromadb.PersistentClient(path="rahman_vector_db")

# =====
# LINE 6: CREATE A CLEAN CORE REGISTRY POOL
# =====
collection = chroma_client.get_or_create_collection(name="rahman_tamil_songs")

# =====
# LINE 7: INITIALIZE THE EMBEDDING BRAIN
# =====
# This pulls down a multilingual model to read and translate your English-script Tamil text
embedding_model = SentenceTransformer("intfloat/multilingual-e5-base")

# =====
# LINE 8: CONVERT FULL LYRICS AND POPULATE DB
# =====
print("\n🔄 Generating vector embeddings. Please wait...")

for index, row in df.iterrows():
    lyric_content = row['Cleaned_Lyrics']

    # Skip processing if a row contains empty lyric values
    if not lyric_content.strip():
        continue

    # Translate text layout into mathematical matrix arrays
    vector_embedding = embedding_model.encode(lyric_content).tolist()

    # Map out metadata using the other 3 columns for game reference
    metadata_package = {
        "title": row['Song Title'],
        "movie": row['Movie/Album'],
        "composer": row['Composer']
    }

    # Securely append inside ChromaDB
    collection.add(
        ids=[f"song_{index}"],
        embeddings=[vector_embedding],
        documents=[lyric_content], # The entire document goes here for robust context search
        metadatas=[metadata_package]
    )

print(f"\n🎉 Complete! Your cleaned database has been embedded successfully.")
print(f"📦 ChromaDB total records stored: {collection.count()} songs.")

```

```

 Missing libraries detected. Installing ChromaDB and Transformers for free...
/usr/local/lib/python3.12/dist-packages/huggingface_hub/utils/_auth.py:112: UserWarning:
The secret `HF_TOKEN` does not exist in your Colab secrets.
To authenticate with the Hugging Face Hub, create a token in your settings tab (https://huggingface.co/settings/tokens), set
You will be able to reuse this secret in all of your notebooks.
Please note that authentication is recommended but still optional to access public models or datasets.
warnings.warn(
Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to enable higher rate limits and fast
WARNING:huggingface_hub.utils._http:Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN t
modules.json: 100%                               387/387 [00:00<00:00, 35.6kB/s]
README.md: 100%                                  179k/179k [00:00<00:00, 12.0MB/s]
sentence_bert_config.json: 100%                  57.0/57.0 [00:00<00:00, 3.95kB/s]
config.json: 100%                                694/694 [00:00<00:00, 55.0kB/s]
model.safetensors: 100%                          1.11G/1.11G [00:21<00:00, 57.8MB/s]
Loading weights: 100%                            199/199 [00:00<00:00, 2495.23it/s]
tokenizer_config.json: 100%                       418/418 [00:00<00:00, 10.4kB/s]
sentencepiece.bpe.model: 100%                     5.07M/5.07M [00:01<00:00, 3.16MB/s]
tokenizer.json: 100%                              17.1M/17.1M [00:00<00:00, 47.9MB/s]
special_tokens_map.json: 100%                    280/280 [00:00<00:00, 25.6kB/s]
config.json: 100%                                200/200 [00:00<00:00, 8.19kB/s]

 Generating vector embeddings. Please wait...

 Complete! Your cleaned database has been embedded successfully.
 ChromaDB total records stored: 154 songs.

```

Double-click (or enter) to edit

```
# =====  
# LINE 1: INSTALL LANGGRAPH & GROQ DRIVERS  
# =====  
!pip install -q langgraph langchain-groq  
  
from typing import TypedDict, List  
import random  
import re  
import chromadb  
from sentence_transformers import SentenceTransformer  
from langgraph.graph import StateGraph, START, END  
from langchain_groq import ChatGroq  
  
# =====  
# LINE 2: INSERT YOUR FREE GROQ KEY HERE  
# =====  
MY_FREE_KEY = "gsk_9680R3tYnWwUyTjSv7Kk339bfEz6DmBtZdVnXfJGguyPzHh"  
  
# 1. Connect to your existing 154-song ChromaDB folder  
chroma_client = chromadb.PersistentClient(path="rahman_vector_db")  
collection = chroma_client.get_collection(name="rahman_tamil_songs")  
embedding_model = SentenceTransformer("intfloat/multilingual-e5-base")  
  
# 2. Connect to the high-speed free Cloud Llama-3.1 model brain  
llm = ChatGroq(model="llama-3.1-8b-instant", groq_api_key=MY_FREE_KEY, temperature=0.4)  
  
class GameState(TypedDict):  
    raw_four_words: str          # Step 1: The exact 4-word phrase pulled from the song  
    ai_clue: str                 # Step 2: The intelligent clue created by Agent 2  
    played_songs: List[str]  
    correct_answer: str  
    movie_name: str  
    user_guess: str  
    attempts: int  
    score: int  
  
# =====  
# STEP 1: THE DATABASE NODE (Pure Song Text)  
# =====  
def fetch_exact_song_phrase_node(state: GameState):  
    """Pulls a song and extracts an exact, contiguous 4-word line from the lyrics."""  
    current_lyrics = state.get("raw_four_words", "love romance rain")  
    played_history = state.get("played_songs", [])  
  
    query vector = embedding_model.encode(current_lyrics).tolist()
```

```

results = collection.query(query_embeddings=[query_vector], n_results=5)

selected_doc = ""
selected_title = ""
selected_movie = ""

# FIX: ChromaDB queries return arrays inside an extra outer list, we unpack the first index [0] safely
if results and 'documents' in results and len(results['documents']) > 0 and len(results['documents'][0]) > 0:
    docs_list = results['documents'][0]
    metadatas_list = results['metadatas'][0]

    for i in range(len(docs_list)):
        title = metadatas_list[i]['title']
        if title not in played_history:
            selected_doc = docs_list[i]
            selected_title = title
            selected_movie = metadatas_list[i]['movie']
            break

# Safety fallback if every matching option was already used or if database parsing hit a glitch
if not selected_doc:
    all_records = collection.get()
    if all_records and len(all_records['documents']) > 0:
        random_index = random.randint(0, len(all_records['documents'])-1)
        selected_doc = all_records['documents'][random_index]
        selected_title = all_records['metadatas'][random_index]['title']
        selected_movie = all_records['metadatas'][random_index]['movie']
    else:
        # Absolute baseline backup if database can't find anything
        selected_doc = "chinna chinna aasai siragadikki aasai\nmuthu muthu aasai"
        selected_title = "Chinna Chinna Aasai"
        selected_movie = "Roja"

# Extract 4 contiguous words exactly as written in a song line
lines = [line.strip() for line in selected_doc.split('\n') if len(line.strip().split()) >= 4]
if lines:
    chosen_line = random.choice(lines)
    words = chosen_line.split()
    exact_phrase = " ".join(words[:4])
else:
    # Fallback if text layout has short lines
    words = selected_doc.split()
    exact_phrase = " ".join(words[:4]) if len(words) >= 4 else " ".join(words)

return {
    "raw_four_words": exact_phrase,
    "correct_answer": selected_title,
    "movie_name": selected_movie,
    "played_songs": played_history + [selected_title],
    "attempts": 0,
    "score": state.get("score", 0)
}

# =====
# STEP 2: AGENT 2 NODE (The Clue Builder)
# =====
def generate_ai_clue_node(state: GameState):
    """Agent 2 reads the real 4-word lyric line and turns it into an engaging clue."""
    phrase = state["raw_four_words"]

    prompt = f"""
    You are an expert Tamil music game host.
    Here is an exact 4-word phrase taken directly from an A.R. Rahman song line:
    "{phrase}"

    Task: Write an engaging, helpful trivia clue for the player based on this phrase.
    Rules:
    - You can translate the phrase to English, describe its mood, or hint at the style.
    - Do NOT mention the song name '{state['correct_answer']}' or movie name '{state['movie_name']}''.
    - Keep your clue concise (1 to 2 short sentences max).

    Your Trivia Clue:"""

    ai_response = llm.invoke(prompt)
    clue_text = ai_response.content.strip()

    print(f"\n🎵 Real Lyric Line: \"{phrase}\"")
    print(f"💡 Agent 2 Clue: {clue_text}")
    return {"ai_clue": clue_text}

def wait_for_user_guess_node(state: GameState):
    guess = input("🎮 Enter your guess (Song Title or Movie Name): ")
    return {"user_guess": guess}

```

```
# =====
# STEP 3: AGENT 3 NODE (The Evaluator Edge)
# =====
def evaluate_guess_edge(state: GameState):
    guess = state["user_guess"].strip().lower()
    answer_title = state["correct_answer"].strip().lower()
    answer_movie = state["movie_name"].strip().lower()
    current_attempts = state["attempts"] + 1
    current_score = state.get("score", 0)

    if guess == answer_title or guess == answer_movie:
        print(f"🎉 Correct! Agent 3 confirms it was '{state['correct_answer']}' from '{state['movie_name']}'".)
        state["score"] = current_score + 1
        return "next_round"
    elif current_attempts >= 3:
        print(f"❌ Game Over! Evaluator reveals: The answer was '{state['correct_answer']}' from '{state['movie_name']}'".)
        print(f"🏆 Final Score: {current_score} points.")
        return "end_game"
    else:
        print(f"❌ Wrong! Evaluator rejects guess. Try again. (Attempts: {current_attempts}/3)")
        state["attempts"] = current_attempts
        return "retry_guess"

# =====
# BUILDING THE COMPILING MAP
# =====
workflow = StateGraph(GameState)

workflow.add_node("fetch_phrase", fetch_exact_song_phrase_node)
workflow.add_node("make_clue", generate_ai_clue_node)
workflow.add_node("get_input", wait_for_user_guess_node)

workflow.add_edge(START, "fetch_phrase")
workflow.add_edge("fetch_phrase", "make_clue")
workflow.add_edge("make_clue", "get_input")

workflow.add_conditional_edges(
    "get_input",
    evaluate_guess_edge,
    {
        "next_round": "fetch_phrase",
        "retry_guess": "get_input",
        "end_game": END
    }
)

game_app = workflow.compile()
print("🌟 New 3-Layer Meaningful Game Engine Compiled Successfully!")
```

137.5/137.5 kB 4.8 MB/s eta 0:00:00

Loading weights: 100%

199/199 [00:00<00:00, 2966.06it/s]

🌟 New 3-Layer Meaningful Game Engine Compiled Successfully!

```
initial_state = {"played_songs": [], "score": 0, "raw_four_words": "rain romance love"}
game_app.invoke(initial_state)
```

```
-----
KeyboardInterrupt                                Traceback (most recent call last)
/tmp/ipykernel_1446/1241278869.py in <cell line: 0>()
      1 initial_state = {"played_songs": [], "score": 0, "raw_four_words": "rain romance love"}
----> 2 game_app.invoke(initial_state)
```

```
-----
32 frames -----
/usr/local/lib/python3.12/dist-packages/torch/nn/modules/linear.py in forward(self, input)
    132     Runs the forward pass.
    133     """
--> 134     return F.linear(input, self.weight, self.bias)
    135
    136     def extra_repr(self) -> str:
```

KeyboardInterrupt:

```
# 1. Compress your ChromaDB folder into a single zip file
!zip -r rahman_vector_db.zip rahman_vector_db
```

```
# 2. Trigger an automatic browser download straight to your computer
from google.colab import files
```

```
files.download("rahman_vector_db.zip")
```

```
adding: rahman_vector_db/ (stored 0%)
adding: rahman_vector_db/chroma.sqlite3 (deflated 63%)
adding: rahman_vector_db/d65c5604-c5b7-4de7-91be-0e6c0bafef1f/ (stored 0%)
adding: rahman_vector_db/d65c5604-c5b7-4de7-91be-0e6c0bafef1f/header.bin (deflated 63%)
adding: rahman_vector_db/d65c5604-c5b7-4de7-91be-0e6c0bafef1f/data_level0.bin (deflated 100%)
adding: rahman_vector_db/d65c5604-c5b7-4de7-91be-0e6c0bafef1f/length.bin (deflated 91%)
adding: rahman_vector_db/d65c5604-c5b7-4de7-91be-0e6c0bafef1f/link_lists.bin (stored 0%)
```

```
# =====
# FINAL PERFECTED ENGINE: SYSTEM INTERCEPTS FOR ACTUAL DATA ACCESS
# =====
!pip install -q langgraph langchain-groq

from typing import TypedDict, List
import random
import chromadb
from sentence_transformers import SentenceTransformer
from langchain_groq import ChatGroq

MY_FREE_KEY = "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"
chroma_client = chromadb.PersistentClient(path="rahman_vector_db")
collection = chroma_client.get_collection(name="rahman_tamil_songs")
embedding_model = SentenceTransformer("intfloat/multilingual-e5-base")
host_llm = ChatGroq(model="llama-3.1-8b-instant", groq_api_key=MY_FREE_KEY, temperature=0.6)

def play_conversational_game():
    all_records = collection.get()
    idx = random.randint(0, len(all_records['documents'])-1)

    hidden_lyrics = all_records['documents'][idx]
    correct_song = all_records['metadatas'][idx]['title']
    correct_movie = all_records['metadatas'][idx]['movie']

    # Clean up the lyrics list array structure
    full_lines_pool = [l.strip() for l in hidden_lyrics.split('\n') if len(l.strip().split()) >= 3]

    # Game trackers
    current_line_index = 0
    chat_history = []
    attempts = 0

    if not full_lines_pool:
        # Fallback security wrap
        full_lines_pool = ["chinna chinna aasai", "siragadikki aasai", "muthu muthu aasai"]

    iconic_line = full_lines_pool[current_line_index]

    print("\n 🎮 [Game Host]: Hello! I am your interactive A.R. Rahman trivia host. Let's have some fun.")
    print(f" 🎵 [Lyric Clue Line]: \"{iconic_line}\"")
    print(" 🗨️ (Type 'next line' for more lyrics, 'give answer' to reveal, 'skip' to change, or chat freely!)")

    while True:
        user_input = input("\n 🗨️ You: ").strip()
        cleanup_command = user_input.lower().strip()

        # --- ORCHESTRATOR INTERCEPT 1: SKIP CONTROL ---
        if cleanup_command in ["skip", "change", "next song", "other song"]:
            print("\n 🔄 [System]: Spinning up a brand new track card...")
            return play_conversational_game()

        # --- ORCHESTRATOR INTERCEPT 2: REAL NEXT LINE FETCHING ---
        if cleanup_command in ["next line", "give next line", "more lyrics"]:
            current_line_index += 1
            if current_line_index < len(full_lines_pool):
                next_lyrics = full_lines_pool[current_line_index]
                print(f"\n 🎮 [Game Host]: You want more text details? Say no more! Here is the actual next line from the c")
                print(f" 🎵 \"{next_lyrics}\"")
                # Update history so AI knows you have more context clue text
                chat_history.append(f"System: Revealed next line: {next_lyrics}")
                continue
            else:
                print("\n 🎮 [Game Host]: Whoops! That is the end of the lyric file sheet. Time to guess!")
                continue

        # --- ORCHESTRATOR INTERCEPT 3: FORCE REVEAL ANSWER ---
        if cleanup_command in ["give answer", "reveal answer", "i give up", "tell me"]:
            print(f"\n 🎮 [Game Host]: Alright, throwing in the towel? No shame in that! The hidden masterpiece was:")
            print(f" 🎵 Song: **{correct_song}** | Movie: **{correct_movie}**")
            print("=====")
```



```

    return play_conversational_game()

# Standard conversational fallback via Llama processing layer
master_system_prompt = f"""
You are a fun game host for an A.R. Rahman music trivia game.
Secret Answer: Song '{correct_song}' from Movie '{correct_movie}'.
Initial clue words shown: "{iconic_line}"
Current incorrect guess count: {attempts}

CRUCIAL INSTRUCTIONS:
1. Never output the exact song '{correct_song}' or movie '{correct_movie}' strings unless the player guesses it right.
2. If the user talks to you or argues about something (e.g., 'this is a Hindi song'), chat back naturally like a human.
3. If they name the correct movie or song, respond with high energy and put 'GAME_WON_MOVE_ON' at the absolute end of the response.

Past Chat Flow Context:
{chat_history[-4:]}

User Input: "{user_input}"
Your Chat Response: ""

response = host_llm.invoke(master_system_prompt)
reply_text = response.content.strip()

if "GAME_WON_MOVE_ON" in reply_text:
    clean_reply = reply_text.replace("GAME_WON_MOVE_ON", "").strip()
    print(f"\n 🎉 [Game Host]: {clean_reply}")
    print("\n=====")
    return play_conversational_game()

print(f"\n 🎙️ [Game Host]: {reply_text}")
chat_history.append(f"User: {user_input}\nHost: {reply_text}")
attempts += 1

```

Loading weights: 100%

199/199 [00:00<00:00, 3793.46it/s]

play_conversational_game()

```

🎙️ [Game Host]: Hello! I am your interactive A.R. Rahman trivia host. Let's have some fun.
🎵 [Lyric Clue Line]: "Shakalaka baby shakalaka baby"
💬 (Type 'next line' for more lyrics, 'give answer' to reveal, 'skip' to change, or chat freely!)

🎙️ [Game Host]: Whoa, what's this? Rachrigan, huh? I love the enthusiasm, but I'm not seeing the connection to the clue we've given.

Current Clue: "Shakalaka baby shakalaka baby"
Incorrect Guess Count: 0

What's your next guess?

🎙️ [Game Host]: You want more text details? Say no more! Here is the actual next line from the composition:
🎵 "Shakalaka baby shakalaka baby"

🎙️ [Game Host]: You want more text details? Say no more! Here is the actual next line from the composition:
🎵 "Shakalaka baby shakalaka baby"

```